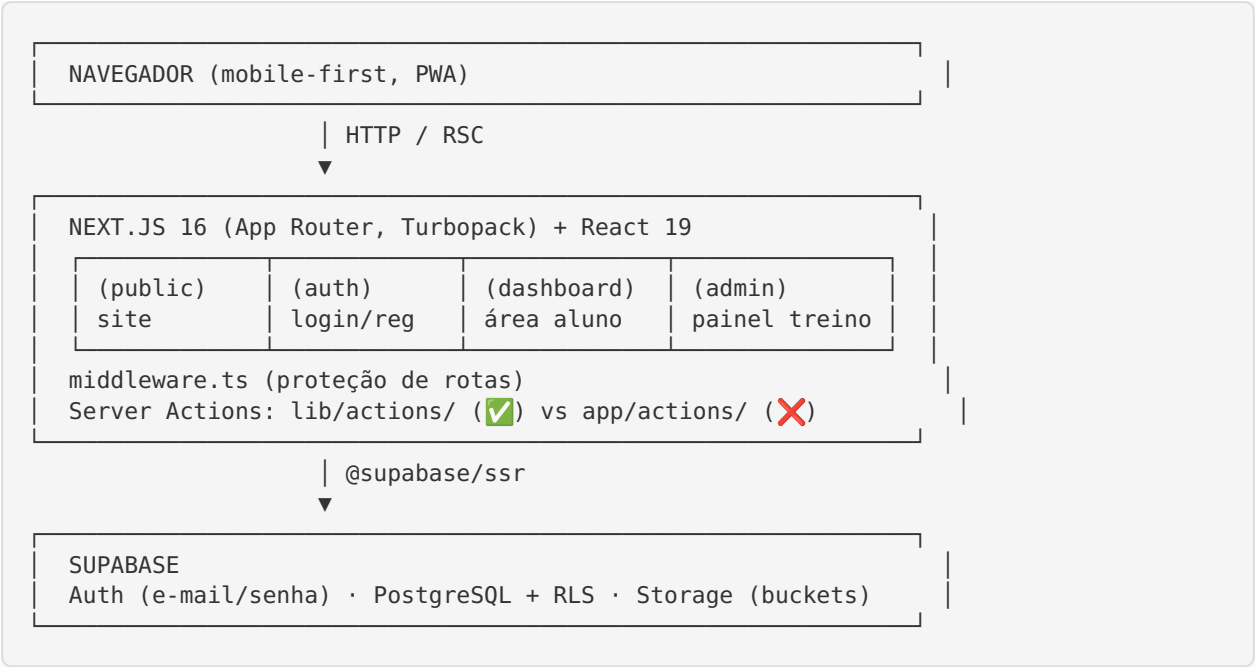


PROJECT_ARCHITECTURE_MAP.md — Mapa Arquitetural

Mapa completo de como o projeto **Born to Run** está organizado hoje: diretórios, rotas, layouts, componentes, integrações, banco de dados, RLS, storage, autenticação e fluxos. Este documento descreve a **realidade atual** (incluindo duplicações e código quebrado), servindo de referência para a reconstrução. Marcações:  correto/funcional ·  parcial/alerta ·  quebrado ·  código morto/duplicado.

1. Visão Geral em Camadas



2. Árvore de Diretórios (raiz)

```

born-to-run/
├─ app/                → App Router (páginas, layouts, actions legadas, estilos)
├─ components/         → Componentes de UI por feature
├─ lib/                → Integrações Supabase, actions corretas, utils
├─ supabase/schema.sql  → Definição do banco (FONTE DA VERDADE)
├─ types/index.ts       → Tipos TypeScript globais
├─ public/             → Assets estáticos (imagens, ícones, manifest)
├─ middleware.ts        → Middleware de auth (ativo; convenção descontinuada no
Next 16)
├─ next.config.ts       → Config mínima do Next
├─ tailwind.config.ts   → 🐛 Config Tailwind v3 (NÃO carregada sob v4)
├─ postcss.config.mjs   → @tailwindcss/postcss (v4)
├─ eslint.config.mjs    → Flat config do ESLint
├─ tsconfig.json        → Config TypeScript (alias @/*)
└─ package.json         → Dependências e scripts

```

3. Páginas, Rotas e Layouts

3.1 Layout raiz

- **app/layout.tsx** ✅ — HTML base, carrega fontes **Inter** (`--font-sans`) e **Barlow Condensed** (`--font-display`), importa `globals.css` , define metadata/manifest.
- **app/globals.css** ⚠️ — Tailwind v4 (`@import 'tailwindcss'`), bloco `@theme inline` com tokens `--color-btr-*` , `:root` com variáveis semânticas, animações e helpers. **Faltam** as classes utilitárias custom usadas nas páginas (`card` , `btn-*` , `badge*` , etc.).

3.2 Grupo (public) — site institucional

Rota	Arquivo	Estado
/	app/(public)/page.tsx	✓ renderiza (usa hex hard-coded)
/sobre	app/(public)/sobre/page.tsx	⚠ classes CSS indefinidas
/equipe	app/(public)/equipe/page.tsx	⚠ classes CSS indefinidas
/galeria	app/(public)/galeria/page.tsx	✗ imagens quebradas/irrelevantes + CSS
/resultados	app/(public)/resultados/page.tsx	⚠ contraste + CSS
/contato	app/(public)/contato/page.tsx	⚠ form sem backend confirmado
Layout	app/(public)/layout.tsx	✓ envolve Header + Footer

- /historia ✗ — não existe (404). A história está embutida em /sobre .

3.3 Grupo (auth) — autenticação

Rota	Arquivo	Estado
/login	app/(auth)/login/page.tsx	⚠ “Esqueceu a senha?” e “Lembrar de mim” mortos
/cadastro	app/(auth)/cadastro/page.tsx	✓ visual; usa client direto
/recuperar-senha	app/(auth)/recuperar-senha/page.tsx	⚠ fluxo de reset quebrado
/recuperar-senha/nova	—	✗ referenciada no reset, não existe
Layout	app/(auth)/layout.tsx	⚠ aspas não escapadas (erro de lint)

3.4 Grupo (dashboard) — área do aluno (⚠️ ROTAS DUPLICADAS)

Feature	Rota linkada na sidebar	Rota alternativa	Observação
Feed	/dashboard ❌ (colunas/FK erradas)	/dashboard/feed ❌ (import PostCard quebra build)	ambas quebradas
Treinos	/treinos ❌ (FK/ treinador)	/dashboard/treinos ✅ (leitura ok)	sidebar aponta p/ quebrada
Perfil	/perfil ⚠️ (usa ProfileForm)	/dashboard/perfil ⚠️ (usa PerfilForm)	dois formulários
Fotos	/fotos ❌ (coluna image_url)	—	quebrada
Layout	app/(dashboard)/lay- out.tsx ⚠️	—	side- bar + logout via / auth/signout (inex- istente)

3.5 Grupo (admin) — painel do treinador

Rota	Arquivo	Estado
/admin	app/(admin)/admin/page.tsx	✅ dashboard com conta- dores
/admin/treinos	app/(admin)/admin/treinos/ page.tsx	✅ CRUD via lib/actions/ad- min.ts
/admin/comunicados	app/(admin)/admin/comunica- dos/page.tsx	✅ CRUD via lib/actions/ad- min.ts
/admin/membros	app/(admin)/admin/membros/ page.tsx	⚠️ promover ok; remover ❌ (sem policy DELETE)
Layout	app/(admin)/layout.tsx	✅ autorização admin server- side; ⚠️ import morto logout

3.6 Rotas especiais

- `app/auth/callback/route.ts` ✅ — troca o código de auth por sessão (callback de e-mail/OAuth do Supabase).
- `/auth/signout` ❌ — referenciada no logout do dashboard, **não existe**.
- `app/favicon.ico` ✅.

4. Componentes

Componente	Caminho	Uso	Estado
Header	components/layout/Header.tsx	público	⚠️ navegação por âncoras; WhatsApp placeholder
Footer	components/layout/Footer.tsx	público	⚠️ CTAs/telefones inconsistentes
PostCard	components/feed/PostCard.tsx	feed	❌ só named export (quebra import default); botões sem handler
CreatePost	components/feed/CreatePost.tsx	feed	💀 concorre com NewPostForm
NewPostForm	components/feed/NewPostForm.tsx	feed	💀 concorre com CreatePost
PerfilForm	components/feed/PerfilForm.tsx	/dashboard/perfil	⚠️ concorre com ProfileForm
ProfileForm	components/profile/ProfileForm.tsx	/perfil	⚠️ usa any (lint)
CreateWorkoutModal	components/workouts/CreateWorkoutModal.tsx	treinos	⚠️ texto de privacidade falso; any
WorkoutCard	components/workouts/WorkoutCard.tsx	treinos	✅ exibição ok
AdminForm	components/admin/AdminForm.tsx	admin	⚠️ classes CSS indefinidas

5. Server Actions (dois conjuntos concorrentes)

5.1 lib/actions/ — ✅ CORRETO (fonte da verdade)

- **auth.ts** : login, signup, logout, resetPassword. Usa lib/supabase/server. (resetPassword tem bug de redirectTo para o domínio do Supabase.)

- **feed.ts** : `createPost` (com `upload p/ post-images`), `toggleLike`, `addComment`, `deletePost`. Colunas corretas (`caption`, `photo_url`, `distance_km`, `duration_minutes`, `pace`). ⚠ **Não são chamadas** por nenhum componente hoje → código correto porém morto.
- **admin.ts** : `createWorkout`, `deleteWorkout`, `createAnnouncement`, `deleteAnnouncement`, `deleteMember` (❌ sem policy DELETE), `toggleAdminRole`.

5.2 app/actions/ — ❌ QUEBRADO

- **post.ts** : insere `content` / `image_url` / `created_by` (colunas inexistentes) → falha em runtime.
- **workouts.ts** : insere `assigned_to` e checa `role === 'treinador'` (coluna/papel inexistentes) → falha.
- **profile.ts** : atualização de perfil (mais próxima do schema, mas parte do conjunto a consolidar).

6. Integração Supabase e Autenticação

6.1 Clients

- **lib/supabase/client.ts** ✅ — client de browser (`createBrowserClient`).
- **lib/supabase/server.ts** ✅ — client de servidor com cookies (`createServerClient`), para RSC/actions.
- **lib/supabase/middleware.ts** 🧑 — `updateSession`, **nunca importado**.

6.2 Middleware ativo (`middleware.ts`)

- Cria client SSR, chama `getUser()` para refrescar a sessão.
- **Protege** `/dashboard` e `/admin` : sem usuário → redireciona para `/login`.
- **Redireciona** usuários logados que acessam `/login` ou `/cadastro` para `/dashboard`.
- ⚠ **Não reforça** `role === 'admin'` em `/admin` (isso só ocorre no layout admin).
- ⚠ Usa a convenção middleware **descontinuada** no Next 16 (migrar para `proxy`).
- `matcher` exclui estáticos e imagens.

6.3 Fluxo de autenticação

Cadastro → `supabase.auth.signUp` → trigger `handle_new_user` cria profile (role='member')
 Login → `supabase.auth.signInWithPassword` → sessão em cookies → redirect `/dashboard`
 Acesso → middleware valida sessão → layout admin valida role='admin'
 Logout → deveria chamar `logout()` de `lib/actions/auth.ts`
 ❌ dashboard usa `POST /auth/signout` (rota inexistente)
 Reset → `resetPassword()` ❌ `redirectTo` aponta p/ domínio Supabase + rota /nova inexistente
 Callback → `app/auth/callback/route.ts` troca code por sessão ✅

7. Banco de Dados

7.1 Tabelas (todas com RLS habilitado)

Tabela	Colunas principais	Relacionamentos
profiles	id, user_id (UNIQUE→auth.users), full_name, avatar_url, bio, cidade, objetivo, role (CHECK member / admin), timestamps	1:1 com auth.users
posts	id, user_id →auth.users, caption, photo_url, dis- tance_km, duration_minutes, pace, timestamps	1:N comments/likes
comments	id, post_id →posts, user_id →auth.users, con- tent (1-500 chars), cre- ated_at	N:1 posts
likes	id, post_id →posts, user_id →auth.users, UNIQUE(post_id,user_id)	N:1 posts
workouts	id, title, description, level (CHECK iniciante/in- termediario/avancado), ob- jective, scheduled_date, created_by →auth.users, timestamps	criado por admin
announcements	id, title, content, cre- ated_by →auth.users, timestamps	criado por admin

⚠ **Não existem** as colunas `posts.content`, `posts.image_url`, nem `workouts.assigned_to`. **Não existe** o papel `'treinador'`. Não há FKs `posts_created_by_fkey` nem `workouts_created_by_fkey` (os vínculos são para `auth.users`, não `profiles`).

7.2 Funções e triggers

- `handle_updated_at()` + triggers `trg_*_updated_at` em `profiles`, `posts`, `workouts`, `announcements`.
- `handle_new_user()` (SECURITY DEFINER) + `trg_on_auth_user_created` → cria profile no signup.
- `is_admin()` (SECURITY DEFINER, STABLE) → usada nas policies.
- Índices em `user_id`, `created_at`, `post_id`, `scheduled_date`, etc.

7.3 Políticas RLS (resumo)

Tabela	SELECT	INSERT	UPDATE	DELETE
profiles	authenticated (true)	(via trigger)	próprio + admin (⚠ admin sem WITH CHECK)	✗ ausente
posts	authenticated	próprio	próprio	próprio ou ad- min
comments	authenticated	próprio	—	próprio ou ad- min
likes	authenticated	próprio	—	próprio
workouts	authenticated (todos veem tudo)	admin	admin	admin
announcements	authenticated	admin	admin	admin

7.4 Storage

- Bucket **avatars** (público, 5MB, jpeg/png/webp): leitura pública; insert/update/delete restritos à pasta do próprio usuário (`foldername[1] = auth.uid()`).
- Bucket **post-images** (público, 10MB): leitura pública; insert por autenticado; delete pelo dono ou admin.

8. Funções Administrativas e Fluxos de Membros

8.1 Painel admin (somente `role='admin'`)

- **Dashboard** (`/admin`): contadores de membros/treinos/comunicados + atalhos.
- **Treinos** (`/admin/treinos`): criar/excluir treino (`createWorkout` / `deleteWorkout`).
- **Comunicados** (`/admin/comunicados`): criar/excluir comunicado.
- **Membros** (`/admin/membros`): promover/rebaixar (`toggleAdminRole` ☒) e remover (`deleteMember`
 ✗ falha silenciosa por falta de policy DELETE).

8.2 Fluxo do membro (aluno)

Cadastro → login → `/dashboard`

- └ Feed da equipe (ver/publicar/curtir/comentar)
- └ Treinos (ver treinos do treinador)
- tudo)
- └ Fotos (galeria pessoal)
- └ Perfil (editar dados + avatar)
- └ Comunicados (ler avisos do treinador)

- ✗ quebrado hoje
- ⚠ sem privacidade (todos veem tudo)
- ✗ quebrado
- ✅ funciona
- ✗ sem tela dedicada

9. Configuração e Build

Arquivo	Papel	Observação
<code>next.config.ts</code>	config do Next	mínimo
<code>postcss.config.mjs</code>	PostCSS	@tailwindcss/postcss (v4)
<code>tailwind.config.ts</code>	Tailwind	👤 formato v3, não carregado
<code>eslint.config.mjs</code>	ESLint	flat config; lint falha (24 problemas)
<code>tsconfig.json</code>	TS	alias <code>@/*</code> ; typecheck falha
<code>middleware.ts</code>	auth	convenção descontinuada
<code>public/manifest.json</code>	PWA	válido; atalhos <code>p/ /dashboard</code> e <code>/dashboard/feed</code>
<code>.env.local</code>	segredos	não versionado; hoje só placeholders
<code>.env.example</code>	exemplo	criado/atualizado nesta branch

10. Resumo do Diagnóstico Arquitetural

- Estrutura de route groups é boa** — organização por contexto (`public / auth / dashboard / admin`) deve ser mantida.
- Duplicação é o vício central** — actions, rotas e componentes em pares (certo/errado). Consolidar é o primeiro passo.
- Banco é o ativo mais forte** — schema coeso e RLS real; precisa apenas de ajustes pontuais (policy DELETE, `WITH CHECK`, decisão sobre privacidade de treinos).
- Camada de apresentação está desalinhada** — a navegação leva às rotas quebradas; o design system (classes custom) não existe de fato.
- Fonte da verdade:** `supabase/schema.sql` + `lib/actions/`. Todo o resto deve convergir para eles.

Fim do PROJECT_ARCHITECTURE_MAP.md.